

CS276

Information Retrieval and Web Search

Christopher D. Manning

(small changes: Maciej Piasecki)

Lecture 12: Naïve BayesText Classification

<https://web.stanford.edu/class/cs276/>

Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze,
Introduction to Information Retrieval, Cambridge University
Press. 2008.

<https://nlp.stanford.edu/IR-book/information-retrieval-book.html>

Is this spam?

From: "" <takworld@hotmail.com>

Subject: real estate is the only way... gem oalvgkay

Anyone can buy real estate with no money down

Stop paying rent TODAY !

There is no need to spend hundreds or even thousands for similar courses

I am 22 years old and I have already purchased 6 properties using the methods outlined in this truly INCREDIBLE ebook.

Change your life NOW !

=====

Click Below to order:

<http://www.wholesaledaily.com/sales/nmd.htm>

=====

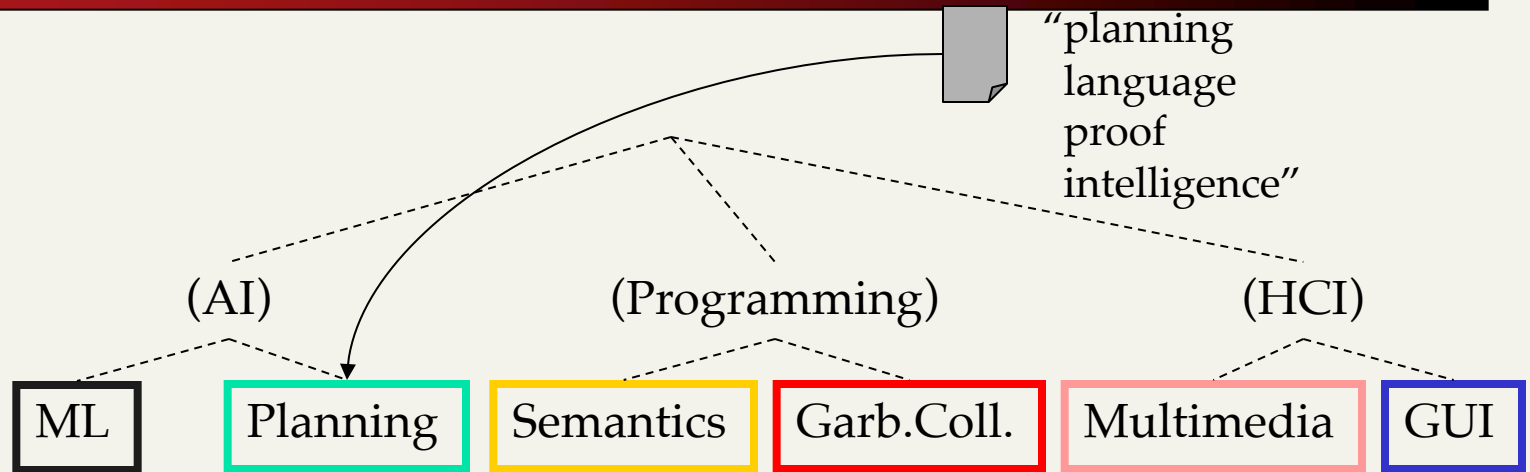
Categorization/Classification

- Given:
 - A description of an instance, $x \in X$, where X is the *instance language* or *instance space*.
 - Issue: how to represent text documents.
 - A fixed set of categories:
$$C = \{c_1, c_2, \dots, c_n\}$$
- Determine:
 - The category of x : $c(x) \in C$, where $c(x)$ is a *categorization function* whose domain is X and whose range is C .
 - We want to know how to build categorization functions (“classifiers”).

Document Classification

*Test
Data:*

Classes:



*Training
Data:*

learning <u>intelligence</u> algorithm reinforcement network...	<u>planning</u> temporal reasoning plan <u>language...</u>	programming semantics <u>language</u> <u>proof...</u>	garbage collection memory optimization region...	...	...
---	--	--	--	-----	-----

(Note: in real life there is often a hierarchy, not present in the above problem statement; and you get papers on ML approaches to Garb. Coll.)

Classification Methods (1)

- Manual classification
 - Used by Yahoo!, Looksmart, about.com, ODP, Medline
 - Very accurate when job is done by experts
 - Consistent when the problem size and team is small
 - Difficult and expensive to scale

Classification Methods (2)

- Automatic document classification
 - Hand-coded rule-based systems
 - One technique used by CS dept's spam filter, Reuters, CIA, Verity, ...
 - E.g., assign category if document contains a given boolean combination of words
 - Standing queries: Commercial systems have complex query languages (everything in IR query languages + accumulators)
 - Accuracy is often very high if a rule has been carefully refined over time by a subject expert
 - Building and maintaining these rules is expensive

Classification Methods (3)

- Supervised learning of a document-label assignment function
 - Many systems partly rely on machine learning (Autonomy, MSN, Verity, Enkata, Yahoo!, ...)
 - k-Nearest Neighbors (simple, powerful)
 - Naive Bayes (simple, common method)
 - SVM - Support-vector machines (newer, more powerful)
 - Deep Learning, e.g. bLSTM neural networks (the newest)
 - ... plus many other methods
 - No free lunch: requires hand-classified training data
 - But data can be built up (and refined) by crowd sourcing
- Note that many commercial systems use a mixture of methods

Bayesian Methods

- Our focus this lecture
- Learning and classification methods based on probability theory.
- Bayes theorem plays a critical role in probabilistic learning and classification.
- Build a *generative model* that approximates how data is produced
- Uses *prior* probability of each category given no information about an item.
- Categorization produces a *posterior* probability distribution over the possible categories given a description of an item.

Bayes' Rule

$$P(C, X) = P(C | X)P(X) = P(X | C)P(C)$$

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}$$

Maximum a posteriori Hypothesis

$$h_{MAP} \equiv \operatorname{argmax}_{h \in H} P(h \mid D)$$

$$= \operatorname{argmax}_{h \in H} \frac{P(D \mid h)P(h)}{P(D)}$$

$$= \operatorname{argmax}_{h \in H} P(D \mid h)P(h)$$

As $P(D)$ is
constant

Maximum likelihood Hypothesis

If all hypotheses are a priori equally likely, we only need to consider the $P(D|h)$ term:

$$h_{ML} \equiv \operatorname{argmax}_{h \in H} P(D | h)$$

Naive Bayes Classifiers

Task: Classify a new instance D based on a tuple of attribute values $D = \langle x_1, x_2, \dots, x_n \rangle$ into one of the classes $c_j \in C$

$$\begin{aligned} c_{MAP} &= \operatorname{argmax}_{c_j \in C} P(c_j \mid x_1, x_2, \dots, x_n) \\ &= \operatorname{argmax}_{c_j \in C} \frac{P(x_1, x_2, \dots, x_n \mid c_j) P(c_j)}{P(x_1, x_2, \dots, x_n)} \\ &= \operatorname{argmax}_{c_j \in C} P(x_1, x_2, \dots, x_n \mid c_j) P(c_j) \end{aligned}$$

Naïve Bayes Classifier:

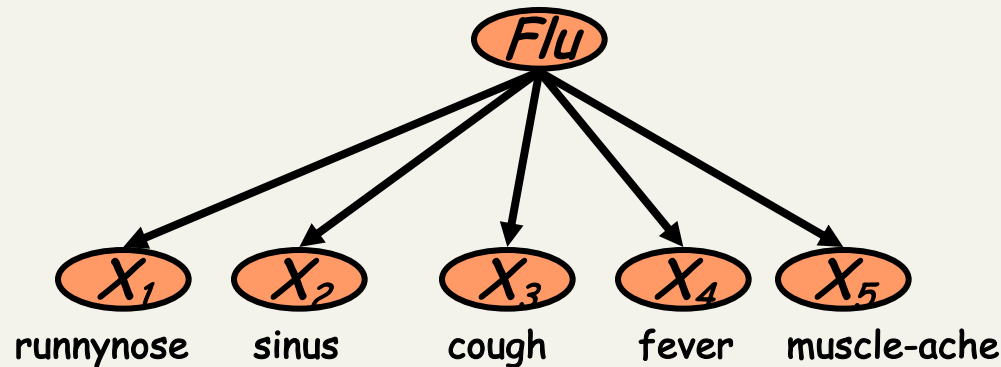
Naïve Bayes Assumption

- $P(c_j)$
 - Can be estimated from the frequency of classes in the training examples.
- $P(x_1, x_2, \dots, x_n | c_j)$
 - $O(|X|^n \cdot |C|)$ parameters
 - Could only be estimated if a very, very large number of training examples was available.

Naïve Bayes Conditional Independence Assumption:

- Assume that the probability of observing the conjunction of attributes is equal to the product of the individual probabilities $P(x_i | c_j)$.

The Naïve Bayes Classifier

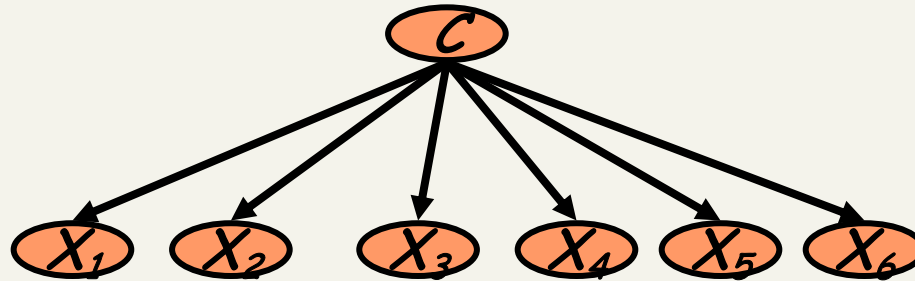


- **Conditional Independence Assumption:**
features detect term presence and are independent of each other given the class:

$$P(X_1, \dots, X_5 | C) = P(X_1 | C) \bullet P(X_2 | C) \bullet \dots \bullet P(X_5 | C)$$

- This model is appropriate for binary variables
 - Multivariate binomial model

Learning the Model

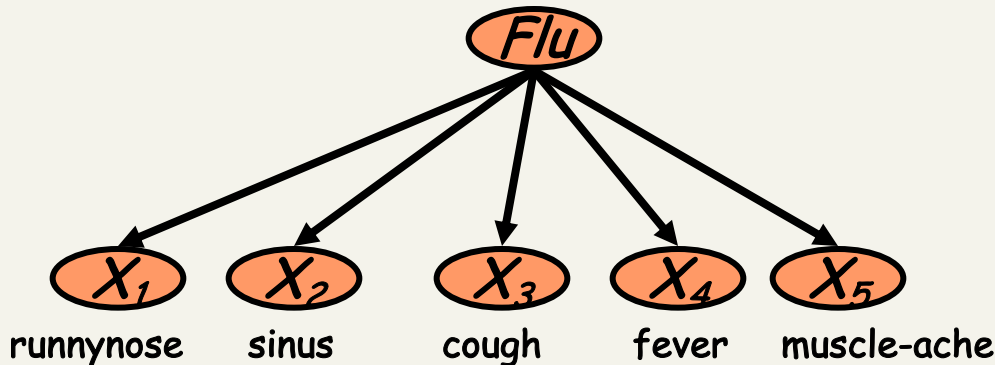


- First attempt: maximum likelihood estimates
 - simply use the frequencies in the data

$$\hat{P}(c_j) = \frac{N(C = c_j)}{N}$$

$$\hat{P}(x_i | c_j) = \frac{N(X_i = x_i, C = c_j)}{N(C = c_j)}$$

Problem with Max Likelihood



$$P(X_1, \dots, X_5 \mid C) = P(X_1 \mid C) \bullet P(X_2 \mid C) \bullet \dots \bullet P(X_5 \mid C)$$

- What if we have seen no training cases where patient had no flu and muscle aches?

$$\hat{P}(X_5 = t \mid C = nf) = \frac{N(X_5 = t, C = nf)}{N(C = nf)} = 0$$

- Zero probabilities cannot be conditioned away, no matter the other evidence!

$$\ell = \arg \max_c \hat{P}(c) \prod_i \hat{P}(x_i \mid c)$$

Smoothing to Avoid Overfitting

$$\hat{P}(x_i | c_j) = \frac{N(X_i = x_i, C = c_j) + 1}{N(C = c_j) + k}$$

of values of X_i

- Somewhat more subtle version

overall fraction in
data where $X_i = x_{i,k}$

$$\hat{P}(x_{i,k} | c_j) = \frac{N(X_i = x_{i,k}, C = c_j) + mp_{i,k}}{N(C = c_j) + m}$$

extent of
“smoothing”

Stochastic Language Models

- Models *probability* of generating strings (each word in turn) in the language (commonly all strings over Σ). E.g., unigram model

Model M

0.2	the						
		the	man	likes	the	woman	
0.1	a	—	—	—	—	—	
0.01	man	0.2	0.01	0.02	0.2	0.01	
0.01	woman						
0.03	said						
0.02	likes						
...							



multiply

$P(s \mid M) = 0.000000008$

Stochastic Language Models

- Model *probability* of generating any string

Model M1		Model M2						
0.2	the	0.2	the	the	class	pleaseth	yon	maiden
0.01	class	0.0001	class	_____	_____	_____	_____	_____
0.0001	sayst	0.03	sayst	0.2	0.01	0.0001	0.0001	0.0005
0.0001	pleaseth	0.02	pleaseth	0.2	0.0001	0.02	0.1	0.01
0.0001	yon	0.1	yon					
0.0005	maiden	0.01	maiden					
0.01	woman	0.0001	woman					

$$P(s|M2) > P(s|M1)$$

Unigram and higher-order models

$P(\text{red yellow red blue})$

■ $= P(\text{red}) P(\text{yellow} | \text{red}) P(\text{red} | \text{red yellow}) P(\text{blue} | \text{red yellow red})$

- Unigram Language Models

$P(\text{red}) P(\text{yellow}) P(\text{red}) P(\text{blue})$

Easy.
Effective!

- Bigram (generally, n -gram) Language Models

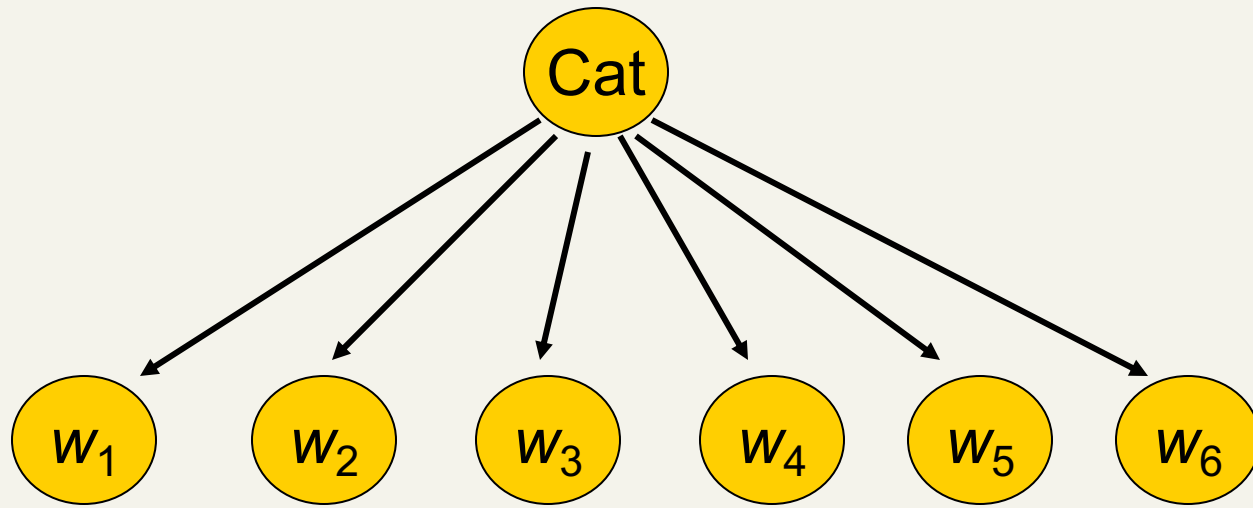
$P(\text{red}) P(\text{yellow} | \text{red}) P(\text{red} | \text{yellow}) P(\text{blue} | \text{red})$

- Other Language Models

- Grammar-based models (PCFGs), etc.

- Probably not the first thing to try in IR

Naïve Bayes via a class conditional language model = multinomial NB



- Effectively, the probability of each class is done as a class-specific unigram language model

Using Multinomial Naive Bayes Classifiers to Classify Text: Basic method

- Attributes are text positions, values are words.

$$\begin{aligned}c_{NB} &= \operatorname{argmax}_{c_j \in C} P(c_j) \prod_i P(x_i | c_j) \\ &= \operatorname{argmax}_{c_j \in C} P(c_j) P(x_1 = \text{"our"} | c_j) \cdots P(x_n = \text{"text"} | c_j)\end{aligned}$$

- Still too many possibilities
- Assume that classification is *independent* of the positions of the words
 - Use same parameters for each position
 - Result is bag of words model (over tokens not types)

Naïve Bayes: Learning

- From training corpus, extract *Vocabulary*
- Calculate required $P(c_j)$ and $P(x_k | c_j)$ terms
 - For each c_j in C do
 - $docs_j \leftarrow$ subset of documents for which the target class is c_j

$$P(c_j) \leftarrow \frac{|docs_j|}{|\text{total \# documents}|}$$

- $Text_j \leftarrow$ single document containing all $docs_j$
- for each word x_k in *Vocabulary*
 - $n_k \leftarrow$ number of occurrences of x_k in $Text_j$

$$P(x_k | c_j) \leftarrow \frac{n_k + \alpha}{n + \alpha |Vocabulary|}$$

Naïve Bayes: Classifying

- positions $\leftarrow$ all word positions in current document which contain tokens found in *Vocabulary*
- Return c_{NB} , where

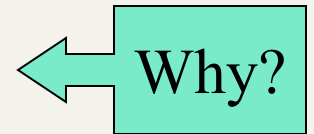
$$c_{NB} = \operatorname{argmax}_{c_j \in C} P(c_j) \prod_{i \in \text{positions}} P(x_i | c_j)$$

Naive Bayes: Time Complexity

- **Training Time:** $O(|D|L_d + |C||V|)$

where L_d is the average length of a document in D .

- Assumes V and all D_i , n_i , and n_{ij} pre-computed in $O(|D|L_d)$ time during one pass through all of the data.
- Generally just $O(|D|L_d)$ since usually $|C||V| < |D|L_d$



- **Test Time:** $O(|C| L_t)$

where L_t is the average length of a test document.

- Very efficient overall, linearly proportional to the time needed to just read in all the data.

Underflow Prevention

- Multiplying lots of probabilities, which are between 0 and 1 by definition, can result in floating-point underflow.
- Since $\log(xy) = \log(x) + \log(y)$, it is better to perform all computations by summing logs of probabilities rather than multiplying probabilities.
- Class with highest final un-normalized log probability score is still the most probable.

$$c_{NB} = \operatorname{argmax}_{c_j \in C} \log P(c_j) + \sum_{i \in \text{positions}} \log P(x_i | c_j)$$

Note: Two Models

- Model 1: Multivariate binomial
 - One feature X_w for each word in dictionary
 - $X_w = \text{true}$ in document d if w appears in d
 - Naive Bayes assumption:
 - Given the document's topic, appearance of one word in the document tells us nothing about chances that another word appears
- This is the model used in the binary independence model in classic probabilistic relevance feedback in hand-classified data (Maron in IR was a very early user of NB)

Two Models

- Model 2: Multinomial = Class conditional unigram
 - One feature X_i for each word pos in document
 - feature's values are all words in dictionary
 - Value of X_i is the word in position i
 - Naïve Bayes assumption:
 - Given the document's topic, word in one position in the document tells us nothing about words in other positions
 - Second assumption:
 - Word appearance does not depend on position

$$P(X_i = w \mid c) = P(X_j = w \mid c)$$

for all positions i, j , word w , and class c

- Just have one multinomial feature predicting all words

Parameter estimation

- Binomial model:

$$\hat{P}(X_w = t \mid c_j) = \begin{array}{l} \text{fraction of documents of topic } c_j \\ \text{in which word } w \text{ appears} \end{array}$$

- Multinomial model:

$$\hat{P}(X_i = w \mid c_j) = \begin{array}{l} \text{fraction of times in which} \\ \text{word } w \text{ appears} \\ \text{across all documents of topic } c_j \end{array}$$

- Can create a mega-document for topic j by concatenating all documents in this topic
- Use frequency of w in mega-document

Classification

- Multinomial vs Multivariate binomial?
 - Multinomial is in general better
 - See results figures later

NB example

- Given: 4 documents
 - D1 (sports): China soccer
 - D2 (sports): Japan baseball
 - D3 (politics): China trade
 - D4 (politics): Japan Japan exports
- Classify:
 - D5: soccer
 - D6: Japan
- Use
 - Add-one smoothing
 - Multinomial model
 - Multivariate binomial model

Feature Selection: Why?

- Text collections have a large number of features
 - 10,000 – 1,000,000 unique words ... and more
- May make using a particular classifier feasible
 - Some classifiers can't deal with 100,000 of features
- Reduces training time
 - Training time for some methods is quadratic or worse in the number of features
- Can improve generalization (performance)
 - Eliminates noise features
 - Avoids overfitting

Feature selection: how?

- Two ideas:
 - Hypothesis testing statistics:
 - Are we confident that the value of one categorical variable is associated with the value of another
 - Chi-square test (χ^2 test)
 - Information theory:
 - How much information does the value of one categorical variable give you about the value of another
 - Mutual information
- They're similar, but χ^2 measures confidence in association, (based on available statistics), while MI measures extent of association (assuming perfect knowledge of probabilities)

χ^2 statistic (CHI)

- χ^2 is interested in $(f_o - f_e)^2 / f_e$ summed over all table entries: is the observed number what you'd expect given the marginals?

$$\chi^2(j, a) = \sum (O - E)^2 / E = (2 - .25)^2 / .25 + (3 - 4.75)^2 / 4.75 + (500 - 502)^2 / 502 + (9500 - 9498)^2 / 9498 = 12.9 \quad (p < .001)$$

- The null hypothesis is rejected with confidence .999,
- since $12.9 > 10.83$ (the value for .999 confidence).

	<i>Term = jaguar</i>	<i>Term $\neq$ jaguar</i>
<i>Class = auto</i>	2 (0.25)	500 (502)
<i>Class $\neq$ auto</i>	3 (4.75)	9500 (9498)

χ^2 statistic (CHI)

There is a simpler formula for 2x2 χ^2 :

$$\chi^2(t, c) = \frac{N \times (AD - CB)^2}{(A + C) \times (B + D) \times (A + B) \times (C + D)}$$

$A = \#(t, c)$	$C = \#(\neg t, c)$
$B = \#(t, \neg c)$	$D = \#(\neg t, \neg c)$

$$N = A + B + C + D$$

Value for complete independence of term and category?

Feature selection via Mutual Information

- In training set, choose k words which best discriminate (give most info on) the categories.
- The Mutual Information between a word, class is:

$$I(w, c) = \sum_{e_w \in \{0,1\}} \sum_{e_c \in \{0,1\}} p(e_w, e_c) \log \frac{p(e_w, e_c)}{p(e_w)p(e_c)}$$

- For each word w and each category c

Feature selection via MI (contd.)

- For each category we build a list of k most discriminating terms.
- For example (on 20 Newsgroups):
 - ***sci.electronics***: circuit, voltage, amp, ground, copy, battery, electronics, cooling, ...
 - ***rec.autos***: car, cars, engine, ford, dealer, mustang, oil, collision, autos, tires, toyota, ...
- Greedy: does not account for correlations between terms
- Why?

Feature Selection

- Mutual Information
 - Clear information-theoretic interpretation
 - May select rare uninformative terms
- Chi-square
 - Statistical foundation
 - May select very slightly informative frequent terms that are not very useful for classification

Feature Selection

(M.P.)

- Heuristics
 - Not based on theoretical foundations
 - But often performing very well
- Just use the commonest terms?
 - In practice, this is often 90% as good
- tf.idf (or tf x idf) term weights
 - very popular, correlated with Pointwise MI
 - term frequency (*tf*)
 - or *wf*, some measure of term density in a doc
 - inverse document frequency (*idf*)
 - measure of informativeness of a term: its rarity across the whole corpus

Feature selection for NB

- In general feature selection is *necessary* for binomial NB.
- Otherwise you suffer from noise, multi-counting
- “Feature selection” really means something different for multinomial NB. It means dictionary truncation
 - The multinomial NB model only has 1 feature
- This “feature selection” normally isn’t needed for multinomial NB, but may help a fraction with quantities that are badly estimated

Feature selection pitfall (M.P.)

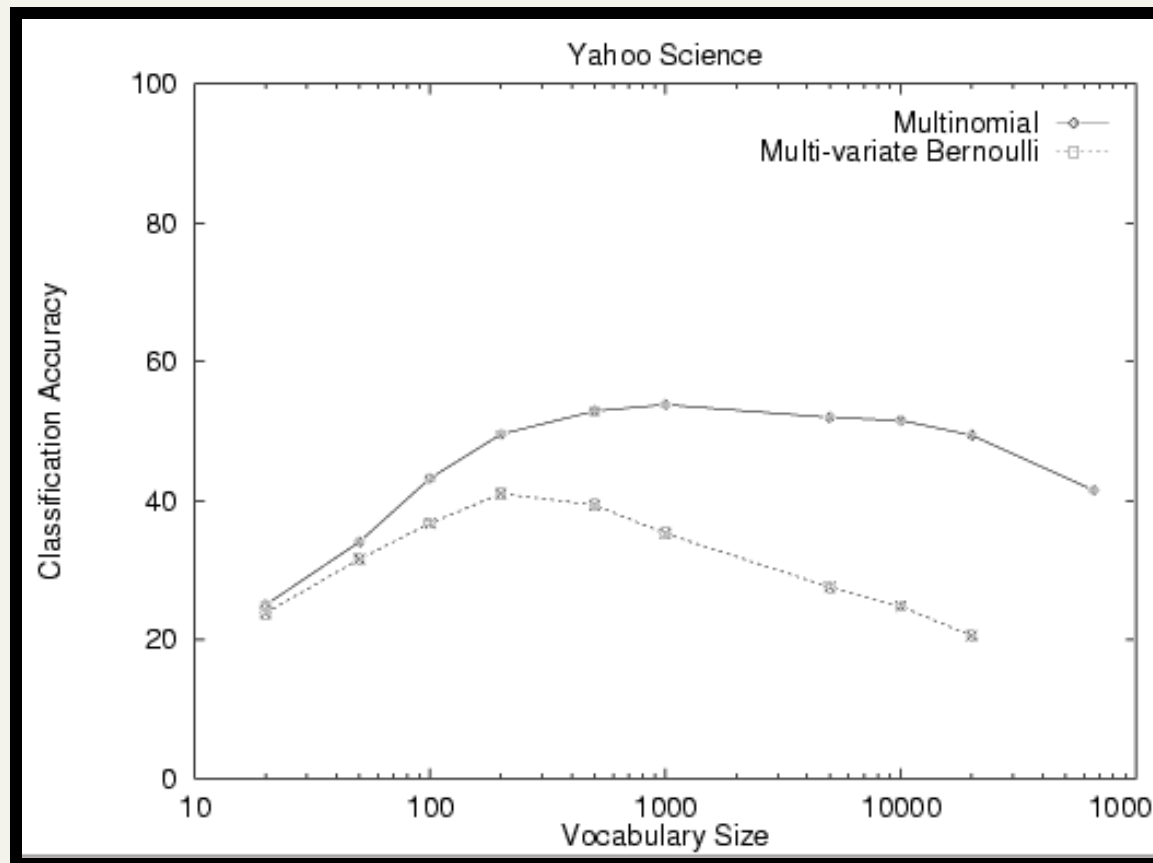
- By selecting features we learn about the domain
 - Selected features show what is important
 - In small collections, they capture accidental dependencies
- Overfitting
 - too much adapting to the test documents
 - feature selection on the whole collection
 - i.e. also on the test documents!!
- Weighting on the whole collection
 - Learning about behaviour of features ALSO from the test documents!!

Evaluating Categorization

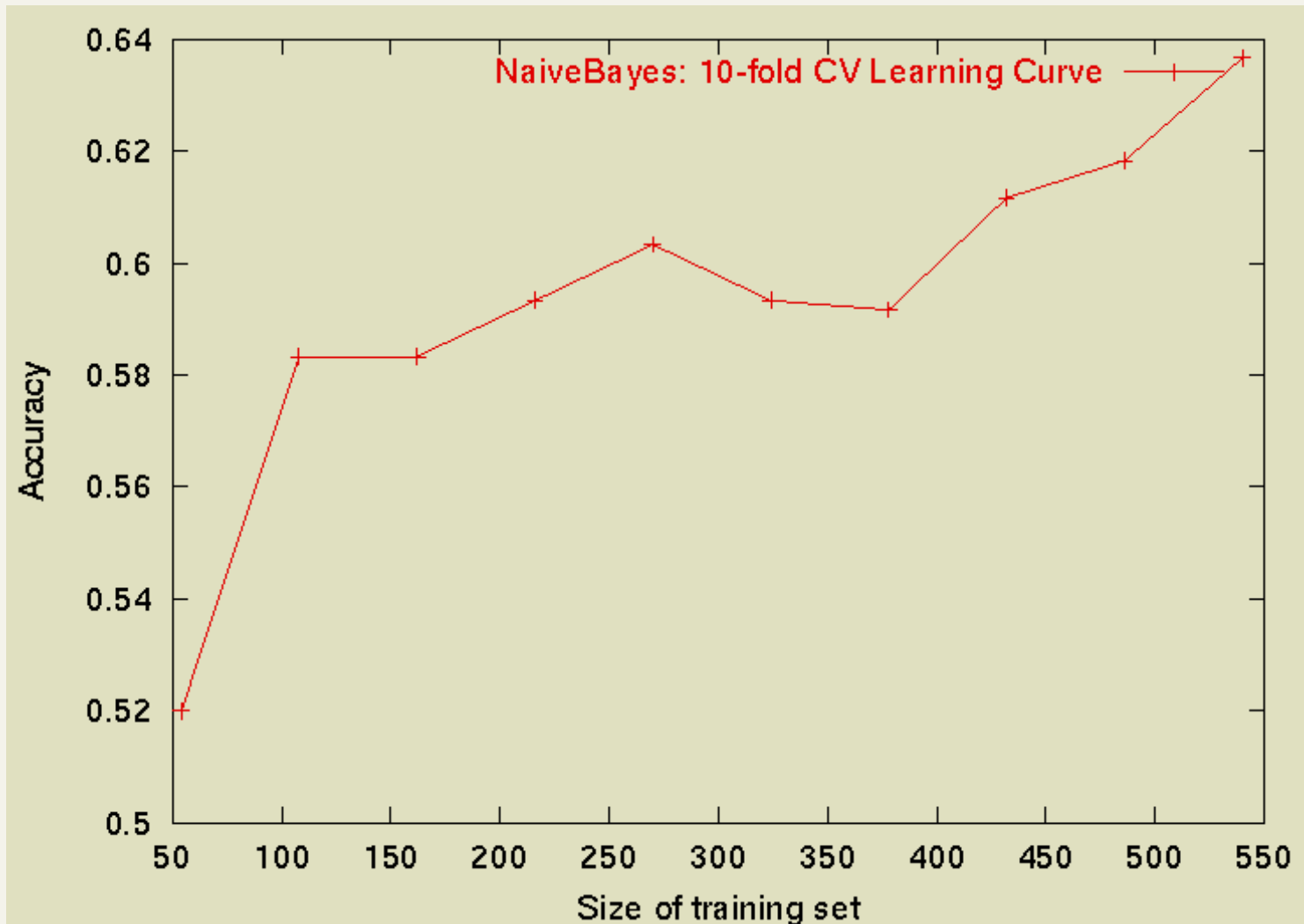
- Evaluation must be done on test data that are independent of the training data (usually a disjoint set of instances).
- *Classification accuracy*: c/n where n is the total number of test instances and c is the number of test instances correctly classified by the system.
- Results can vary based on sampling error due to different training and test sets.
- Average results over multiple training and test sets (splits of the overall data) for the best results.

Example: AutoYahoo!

- Classify 13,589 Yahoo! webpages in “Science” subtree into 95 different topics (hierarchy depth 2)



Sample Learning Curve (Yahoo Science Data): need more!



WebKB Experiment

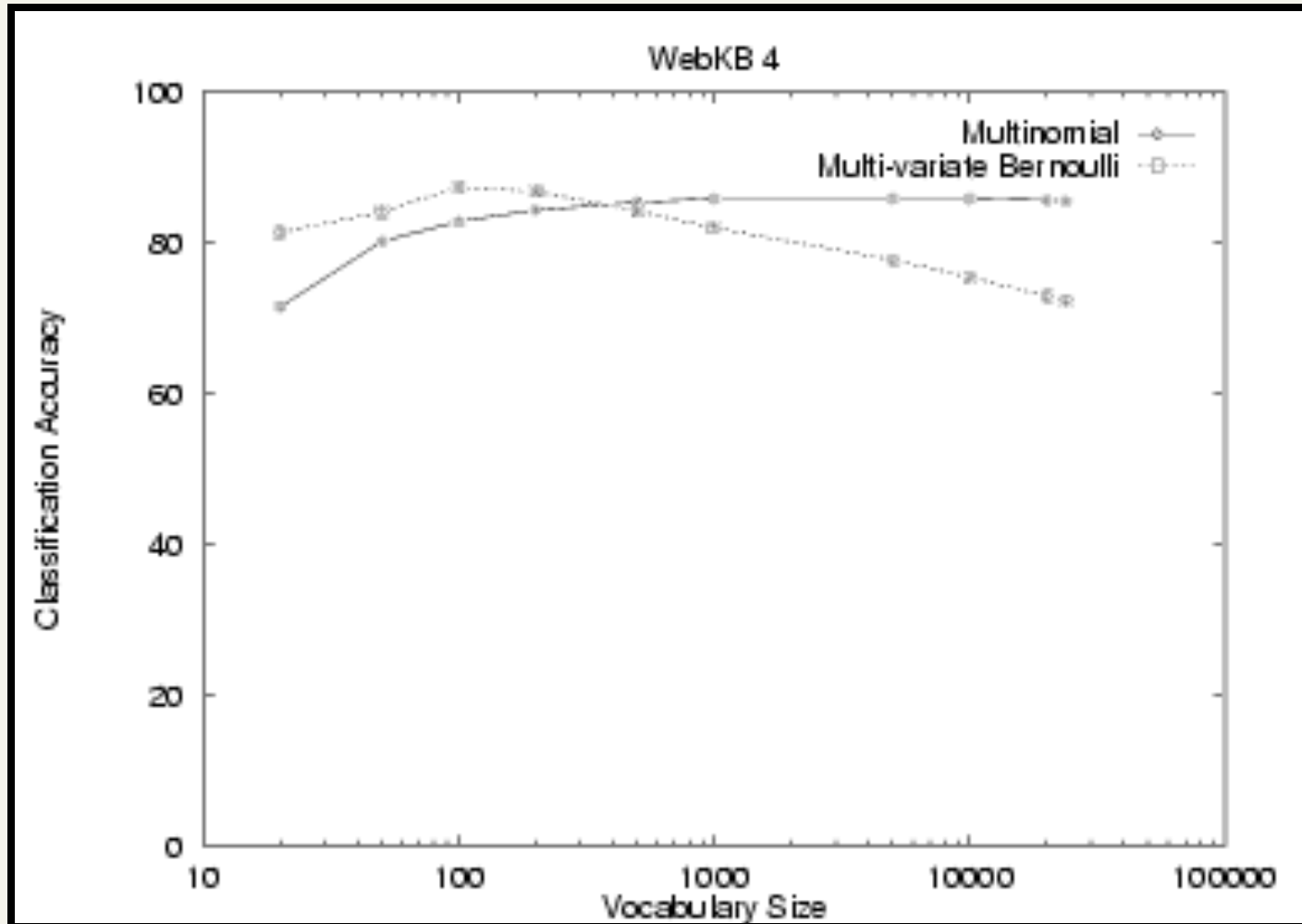
- Classify webpages from CS departments into:
 - student, faculty, course, project
- Train on ~5,000 hand-labeled web pages
 - Cornell, Washington, U.Texas, Wisconsin
- Crawl and classify a new site (CMU)

■ Results:

	Student	Faculty	Person	Project	Course	Department
Extracted	180	66	246	99	28	1
Correct	130	28	194	72	25	1
Accuracy:	72%	42%	79%	73%	89%	100%



NB Model Comparison



Faculty

associate	0.00417
chair	0.00303
member	0.00288
ph	0.00287
director	0.00282
fax	0.00279
journal	0.00271
recent	0.00260
received	0.00258
award	0.00250

Students

resume	0.00516
advisor	0.00456
student	0.00387
working	0.00361
stuff	0.00359
links	0.00355
homepage	0.00345
interests	0.00332
personal	0.00332
favorite	0.00310

Courses

homework	0.00413
syllabus	0.00399
assignments	0.00388
exam	0.00385
grading	0.00381
midterm	0.00374
pm	0.00371
instructor	0.00370
due	0.00364
final	0.00355

Departments

departmental	0.01246
colloquia	0.01076
epartment	0.01045
seminars	0.00997
schedules	0.00879
webmaster	0.00879
events	0.00826
facilities	0.00807
eople	0.00772
postgraduate	0.00764

Research Projects

investigators	0.00256
group	0.00250
members	0.00242
researchers	0.00241
laboratory	0.00238
develop	0.00201
related	0.00200
arpa	0.00187
affiliated	0.00184
project	0.00183

Others

type	0.00164
jan	0.00148
enter	0.00145
random	0.00142
program	0.00136
net	0.00128
time	0.00128
format	0.00124
access	0.00117
begin	0.00116

Violation of NB Assumptions

- Conditional independence
- “Positional independence”
- Examples?

Naïve Bayes Posterior Probabilities

- Classification results of naïve Bayes (the class with maximum posterior probability) are usually fairly accurate.
- However, due to the inadequacy of the conditional independence assumption, the actual posterior-probability numerical estimates are not.
 - Output probabilities are generally very close to 0 or 1.

When does Naive Bayes work?

Sometimes NB performs well even if the Conditional Independence assumptions are **badly** violated.

Classification is about predicting the correct class label and NOT about accurately estimating probabilities.

Assume two classes c_1 and c_2 . A new case A arrives.

NB will classify A to c_1 if:

$$P(A, c_1) > P(A, c_2)$$

	$P(A, c_1)$	$P(A, c_2)$	Class of A
Actual Probability	0.1	0.01	c_1
Estimated Probability by NB	0.08	0.07	c_1

Besides the big error in estimating the probabilities the classification is still **correct**.

Correct estimation $\Rightarrow$ accurate prediction

but **NOT**

~~accurate prediction $\Rightarrow$ Correct estimation~~